Tarjan’s Strongly Connected Components (SCC)

1. Definition

A Strongly Connected Component (SCC) is a maximal group of vertices in a directed graph where every vertex can reach every other vertex by following directed edges. Tarjan’s algorithm finds all SCCs in one Depth-First Search (DFS) traversal.

2. Important Terms

Discovery time (disc[u]): The time when vertex u is first visited by DFS.

Low-link value (low[u]): The smallest discovery time reachable from u through DFS-tree edges and, when applicable, a back edge to a vertex still on the stack.

Stack: Stores vertices that have been visited but have not yet been assigned to a completed SCC.

3. Algorithm Steps

1. Start DFS from each unvisited vertex.

2. When visiting vertex u, assign disc[u] = low[u] = current time and push u onto the stack.

3. For each outgoing neighbour v: if v is unvisited, run DFS on v and update low[u] = min(low[u], low[v]); if v is still on the stack, update low[u] = min(low[u], disc[v]).

4. After processing neighbours, if low[u] = disc[u], u is the root of an SCC.

5. Pop vertices from the stack until u is popped. All popped vertices form one SCC.

6. Continue until every vertex has been visited.

4. Example

Consider the directed edges: A → B, B → C, C → A, B → D, D → E, and E → D.

The SCCs are {A, B, C} and {D, E}. A, B, and C can all reach one another; D and E can reach one another. There is no path from D or E back to A, B, or C.

5. Pseudocode

```
TARJAN(u):
disc[u] = low[u] = time
time = time + 1
push u onto stack
onStack[u] = true

for each neighbour v of u:
if v is unvisited:
TARJAN(v)
low[u] = min(low[u], low[v])
else if onStack[v]:
low[u] = min(low[u], disc[v])

if low[u] == disc[u]:
repeat:
v = pop stack
onStack[v] = false
output v as part of current SCC
until v == u
```

6. Time and Space Complexity

Complexity	Value	Reason
Time	O(V + E)	Each vertex and edge is processed a constant number of times.
Space	O(V)	Discovery/low arrays, stack, and recursion stack.

7. Applications

• Detecting groups of mutually reachable pages or accounts.

• Analyzing dependencies in directed graphs.

• Finding cycles and simplifying graphs by contracting SCCs.

• Program analysis and network analysis.

8. Key Points to Remember

• Tarjan’s algorithm uses DFS and a stack.

• Only vertices still on the stack are considered for the back-edge low-link update.

• When low[u] equals disc[u], u is the root of an SCC.

• The algorithm runs in O(V + E) time.

Conclusion: Tarjan’s algorithm efficiently identifies all strongly connected components of a directed graph in O(V + E) time using DFS discovery times, low-link values, and a stack.